

SYSTEM AND PRODUCT REVIEW

NEST ecommerce readiness audit

Product experience, commerce capabilities, architecture, security, reliability, quality and delivery roadmap.

RELEASE
VERDICT

Conditional pilot only. Strong commerce foundation; not ready for a broad production launch.

AUDIT DATE 3 August 2026

SYSTEM NestJS 11 + Prisma/PostgreSQL + Next.js 16 / React 19

SCOPE Repository, data model, APIs, storefront, administration, controls, tests, build and external benchmarks

METHOD Static review + executable verification + dependency audit + primary-source research

Headline finding

The right near-term architecture is a hardened modular monolith, not microservices. The most valuable work is completing missing commerce domains and production controls while preserving the existing transactional core.

Prepared from the working tree as observed. Uncommitted changes were treated as part of the system under review.

01 Executive verdict

A compact answer to: is this a modern, standard and reliable online shop?

OVERALL
MATURITY

62/100 - credible MVP / controlled pilot. No-go for broad public launch until P0 blockers are closed.

Product experience 61/100



Commerce capability 48/100



Architecture 72/100



Security and privacy 55/100



Reliability and operations 58/100



Quality engineering 76/100



What is already good

- Atomic stock reservation prevents overselling under concurrent checkout.
- Idempotency keys, durable confirmation tokens and immutable order-line price/name snapshots are sound foundations.
- RBAC is granular across catalog, inventory, orders, shipping, promotions, moderation, analytics and audit.
- PostgreSQL migrations, constraints, transaction tests, request IDs, health probes, redacted logs and CI gates are present.
- The storefront covers discovery, product detail, favorites, reviews, cart, delivery checkout and confirmation with coherent interaction states.

Why it is not production-ready

- The commercial model is COD-only and lacks payment records, tax, refunds, returns, exchanges and reconciliation.
- Password reset has no production delivery path; order email has no durable retry or outbox.
- The dependency audit reports five high-severity findings across the production dependency trees.
- No verified backup/restore, disaster recovery objectives, SLOs, metrics/alerts, deployment topology or capacity evidence exists.
- The storefront is client-data-fetch heavy, small-catalog oriented and lacks product structured data, per-product metadata, sitemap/robots and field performance telemetry.

02 Scope, evidence and limitations

Evidence reviewed

Area	Evidence
Code and structure	~20,106 TypeScript/TSX lines; 23 application routes; Nest modules; Prisma schema; eight migrations; CI workflow; local run policy.
Executable checks	Lint passed; backend and dashboard type checks passed; Prisma schema valid; production build passed.
Automated tests	12 backend unit tests, 9 component tests, 18 API integration tests, 3 database tests and 1 browser E2E test passed.
Dependency security	npm audit --omit=dev executed for backend and dashboard. Findings are discussed in section 07.
External benchmark	PCI SSC, OWASP, W3C WCAG 2.2, Next.js, Google Search/Web Vitals/SRE, AWS Reliability, Stripe and Baymard original research.

Important limitation

VISUAL AUDIT

The in-app browser was unavailable. No manual visual click-through, responsive-device inspection, Lighthouse run or assistive-technology test was performed. Product findings use source, tests, route output and successful E2E execution. Visual and accessibility conformance remain unverified.

Interpretation

Scores are an evidence-based maturity model created for this review, not a certification. Security findings are architectural observations and dependency scan results, not a penetration test. Legal and tax requirements depend on launch countries and business model.

03 Product and customer experience

Current journey

Journey stage	Present	Assessment
Discovery	Hero, featured products, search, category filters, sort, pagination	Good small-catalog UX; all products are fetched and filtered in-browser, which will not scale.
Product detail	Gallery, availability, quantity, favorites, comments, material/dimensions/care	Solid essentials; no variants/SKU, delivery estimate by address, return promise, related items or rich SEO.
Bag	Persistent local cart, quantity limits, stock-aware controls	Simple and understandable; guest cart is device-only and stores stale product snapshots.
Checkout	Guest-capable contact/address, shipping method, promotion, COD	Low friction, but no tax, address verification, promotion preview, payment choices or policy/help links.
Post-purchase	Durable confirmation and signed-in order history	Useful foundation; no cancellation request, returns, refunds, support case or self-service tracking flow.

Product-specific observations

- Nested categories are supported in the API, but the storefront filters by direct category slug after downloading all products. Parent-category behavior can therefore diverge from the backend.
- Promotion value is calculated only when an order is placed. The button shows a pre-discount total, so the shopper cannot validate eligibility and final amount before commitment.
- Checkout drafts include email, phone and address in localStorage. This improves recovery but creates shared-device privacy risk and has no expiry, consent or clear-data control.
- Comments publish immediately for any signed-in account. Moderation exists after publication, but verified-purchase status, spam controls and pre-moderation are absent.
- The product grid and product detail are Client Components that fetch after hydration. This weakens first-load resilience, rich metadata and crawler-visible product content.

PRODUCT RECOM
MENDATION

Keep the clear visual language and short checkout. Move catalog reads and product detail content to Server Components, keep only interactive controls client-side, and add server-side search/filter/pagination plus transparent landed-cost preview.

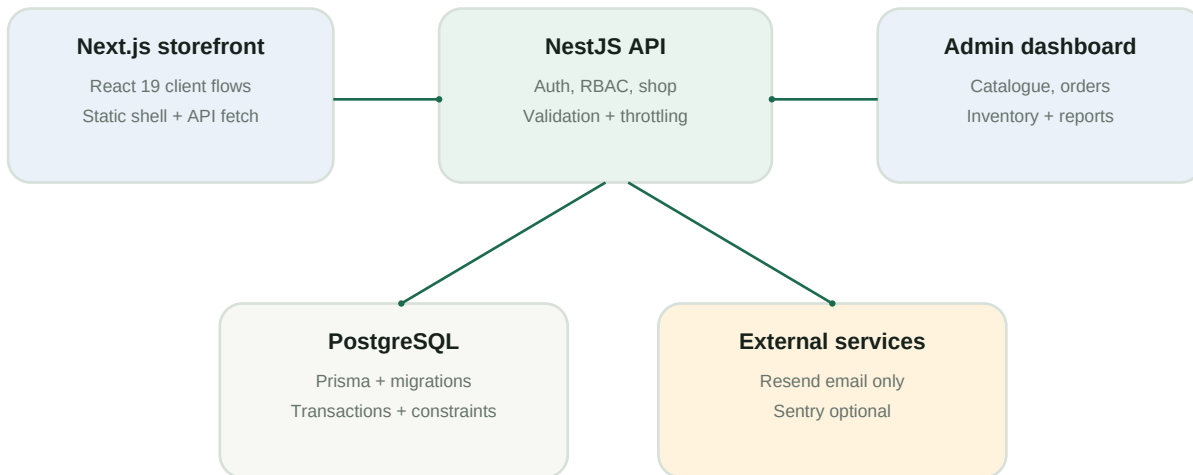
04 Commerce capability model

Domain	Maturity	Gap to a reliable shop
Catalog	Partial	Categories, images, content and stock exist. Add SKU, variants/options, barcode, weight, structured dimensions, brand, cost, compare-at price, channels and publish scheduling.
Pricing	Partial	Server-calculated price snapshots and promotions exist. Add price lists, currency rules, tax-inclusive/exclusive policy and promotion preview/stacking rules.
Inventory	Strong MVP	Atomic reservation, movements and cancellation restock are good. Add reservation expiry, multi-location stock, cycle counts and reconciliation.
Checkout	Partial	Guest COD checkout is coherent. Add address validation, tax, fraud controls, shipping zones/quotes, explicit terms and final price confirmation.
Payments	Missing	No Payment/Transaction model, authorization/capture, webhook ledger, failure recovery, reconciliation or PCI scope decision.
Fulfilment	Partial	Status transitions, notes and shipments exist. Add split fulfilment semantics, labels, carrier events, delivery exceptions and SLA queues.
Returns/refunds	Missing	No return merchandise authorization, return reasons, restock disposition, refund ledger or exchange flow.
Customer service	Early	Order history and internal notes exist. Add customer-visible timeline, contact/support cases, cancellation/return requests and consent history.
Growth/SEO	Early	Favorites, reviews and promotions exist. Add structured product data, Merchant Center feed, sitemap, canonical URLs and analytics funnel events.

BUSINESS DECISION

If NEST is intentionally a local COD-only pilot, payments and PCI can be deferred. They cannot be omitted from a claim of modern full ecommerce readiness.

05 Architecture assessment



Missing production plane: worker/queue, payments, tax, backups, metrics, CDN/object storage

Verdict: retain the modular monolith

The current scale does not justify microservices. Nest modules and a single transactional PostgreSQL database are the correct reliability/cost tradeoff. The improvement path is stronger boundaries and asynchronous work, not network fragmentation.

Architectural strengths

- Clear modules for auth, RBAC, shop, audit, system logs, health and Prisma.
- Database transactions protect order creation, stock reservation, promotion redemption and cancellation restock.
- Order lines snapshot product name and unit price; shipping and promotion details are also copied to the order.
- Explicit DTO validation with whitelist and forbidden unknown properties reduces mass-assignment risk.

Architectural liabilities

- ShopService combines catalog, reviews, favorites, checkout, fulfilment, inventory, shipping, promotions and analytics in one large service. Split by domain use case while keeping one deployable.
- Email is invoked synchronously after order commit. A crash between commit and notification-state update can strand work; there is no durable retry/outbox.
- RBAC default synchronization deletes and recreates role-permission links at application startup. It is non-atomic and risky with multiple replicas.
- Rate limiting is process-local; REDIS_URL is validated but unused. Multiple replicas would enforce inconsistent limits.
- Starter catalog and shipping data are mutated during module initialization, mixing runtime boot with seeding/migration responsibilities.

06 Data integrity and order lifecycle

What is reliable today

Control	Evidence	Assessment
Oversell prevention	Conditional Product.updateMany where stockQty >= quantity inside transaction	Strong concurrency primitive; tested.
Retry safety	Unique idempotencyKey plus duplicate recovery	Good. Also compare full canonical request fingerprint, not only selected fields.
Promotion race	Optimistic usedCount update within order transaction	Good MVP; add per-customer limits and explicit stacking policy.
Status integrity	Allowed transition map plus compare-and-update	Good. Shipment status and order status can still drift independently.
Auditability	InventoryMovement, OrderStatusEvent, AuditLog, SystemLog	Strong foundation; define retention and immutability expectations.

Schema gaps

- No database CHECK constraints enforce non-negative stock/prices/totals, discount bounds, rating range or coherent timestamps. Validation exists mostly in application code.
- Money uses PostgreSQL INTEGER. This is acceptable for a small single-currency store but needs explicit overflow bounds and currency consistency; BIGINT is safer for aggregation-heavy multi-market growth.
- Shipping address is unstructured JSON. It is flexible but hard to validate, index, redact, migrate and retain selectively.
- No cart, payment, tax line, refund, return, webhook event, consent or customer support entities exist.
- Inventory is a single stockQty per product; there are no warehouses, allocated/available distinction or reservation expiry.

PRIORITY DATA
CHANGE

Introduce Payment, PaymentEvent, TaxLine, Return, Refund and OutboxEvent tables before online payments. Make external event IDs unique and handlers idempotent.

07 Security, privacy and dependency risk

Controls already present

- bcrypt cost 12, short-lived access tokens, rotated hashed refresh tokens, HttpOnly refresh cookie and refresh revocation on password change.
- Helmet, explicit CORS allow-list, input validation, global throttling, tighter auth throttling and redacted authorization/cookie logs.
- Permission checks on administrative commerce routes and object ownership checks for customer order history/comments.

Release blockers and hardening

Priority	Risk	Required action
P0	Dependency advisories	Backend: @nestjs/swagger -> js-yaml 5.2.1. Frontend: Next -> postcss 8.4.31 and sharp 0.34.5. npm audit classified 2 + 3 high findings. Upgrade and rerun tests/build/audit.
P0	Account recovery	Password reset creates a token but sends no production email. Implement single-use delivery, abuse controls and operational observability.
P0	PII governance	Shipping/contact PII lacks documented retention, deletion/export, access review, masking and backup handling policy.
P1	Identity hardening	Add email verification, MFA for staff, issuer/audience validation, refresh-family reuse response and session/device management.
P1	Password storage	bcrypt 12 is acceptable but OWASP now prefers Argon2id. Add a 72-byte maximum while bcrypt remains, weak-password screening and migration-on-login.
P1	Abuse prevention	Use shared rate limits, checkout/review-specific controls, bot defenses, promotion abuse controls and verified-purchase reviews.
P1	Web security	Add an explicit storefront CSP, security.txt, dependency scanning gate, secret scanning and periodic access review.

Context

The exact exploitability of transitive advisories depends on runtime paths. They are still unacceptable as unresolved release findings. Do not run npm audit fix --force blindly; upgrade the owning packages, inspect lockfile changes and execute the complete quality suite.

08 Reliability and operations

Present

- Liveness and database readiness endpoints, graceful Prisma shutdown, request IDs, structured logs and optional Sentry capture.
- Dockerized local PostgreSQL with health checks; repeatable migrations; CI executes lint, types, tests, build and E2E.

Missing for production

Dimension	Gap
Availability	No deployment architecture, load balancer, autoscaling, zero-downtime migration strategy or multi-zone database evidence.
Durability	No backup schedule, point-in-time recovery, restore drill, RPO/RTO or data export evidence.
Observability	No metrics, traces, dashboards or actionable alerts. Database-backed system logs can amplify database incidents.
Async resilience	No queue, dead-letter handling, retries or outbox for email/payment/carrier integrations.
Performance	No load test, query plan review, CDN strategy, image ownership pipeline, bundle budget or field Core Web Vitals.
Incident response	No runbooks, on-call ownership, escalation, rollback procedure, status communication or post-incident practice.

Recommended service objectives

SLI	Initial internal objective
Storefront availability	99.9% successful synthetic browse + product detail over 30 days
Checkout correctness	99.99% of accepted orders have matching totals, inventory movements and status event
Checkout latency	p95 order-create API under 750 ms excluding external payment redirect
Notification freshness	99% of confirmations sent or queued within 2 minutes
Recovery	RPO <= 15 minutes; RTO <= 60 minutes, proven by quarterly restore drill

09 Quality engineering and maintainability

VERIFIED STATUS

All executed code-quality, unit, component, integration, database, production-build and existing E2E checks passed on 3 August 2026.

Layer	Result	Coverage signal
Lint + types	PASS	Backend and dashboard
Unit	12 PASS	Auth, DTO/environment and RBAC rules
Frontend component	9 PASS	Auth, dashboard and order confirmation
API integration	18 PASS	Auth, health, order integrity, catalog, RBAC and analytics
Database	3 PASS	PostgreSQL constraints, transactions and relations
Browser E2E	1 PASS	Registration/profile/sign-out/sign-in only
Production build	PASS	Nest build + 23 Next routes

Material testing gaps

- No browser E2E for browse -> product -> cart -> checkout -> confirmation, nor admin fulfilment.
- No automated accessibility suite, keyboard/screen-reader validation, visual regression or responsive matrix.
- No payment/webhook contract tests because the domain does not exist.
- No load/soak testing, chaos/recovery tests, restore drill or migration rollback rehearsal.
- No coverage thresholds or mutation testing; pass counts should not be mistaken for broad behavioral coverage.

Maintainability

The modular structure is healthy, but ShopService and several dashboard management components are becoming high-change concentration points. Split by use case/domain, introduce repositories only where they clarify contracts, and retain transaction orchestration in application services.

10 Prioritized 30/60/90-day roadmap

Window	Theme	Deliverables
0-30 days	Release safety	Upgrade vulnerable dependency paths; implement production password-reset email; add storefront CSP; add PII retention/masking policy; add shopping and admin E2E; define launch market and COD/payment scope.
0-30 days	Product trust	Add privacy, terms, shipping and returns pages; promotion preview; final landed-cost confirmation; verified-purchase rule or pre-moderation for reviews.
31-60 days	Architecture	Split ShopService into Catalog, Checkout, Order, Inventory, Promotion and Fulfilment application services; move seed/bootstrap mutations to explicit commands; make RBAC synchronization transactional.
31-60 days	Reliability	Add transactional outbox + worker, retry/dead-letter policy, shared Redis throttling, metrics/traces, SLO dashboards and alerting.
31-60 days	Storefront	Server-render catalog/product data; server-side filter/search/page; dynamic product metadata; Product/Offer structured data; sitemap/robots/canonicals; Core Web Vitals reporting.
61-90 days	Commerce completion	Add hosted PSP checkout, signed idempotent webhooks, payment ledger, tax policy/service, refund/return model and reconciliation jobs.
61-90 days	Operations	Production topology, managed PostgreSQL PITR, object storage/CDN, recovery drill, load test, capacity budget, incident runbooks and staged rollout.

Recommended target architecture

- One deployable modular monolith for transactional commerce APIs.
- One Next.js storefront using Server Components for catalog content and small client islands for cart/favorites.
- Managed PostgreSQL as system of record; Redis for distributed limits/cache/queue coordination.
- Worker process consuming transactional outbox events for email, payment, carrier and analytics side effects.
- Hosted payment page to minimize payment-data exposure; signed, deduplicated asynchronous webhook handlers.
- Object storage + CDN for owned product images; search engine only when catalog size/relevance evidence justifies it.

ARCHITECTURE
PRINCIPLE

Do not introduce microservices yet. Earn each service boundary through scale, ownership or isolation needs; transactions are currently a competitive reliability advantage.

11 Production launch gate

Gate	Requirement	Observed state
BLOCK	No unresolved high production dependency audit findings	Currently fails
BLOCK	Customer can recover account through a tested production email flow	Currently fails
BLOCK	Checkout/payment/tax/returns scope matches the public commercial promise	Currently COD-only
BLOCK	Backups, point-in-time recovery and restore drill meet approved RPO/RTO	No evidence
BLOCK	Order notifications and future payment events have durable retry/outbox	No evidence
BLOCK	Privacy, retention, deletion/export and staff PII access policies approved	No evidence
REQUIRED	Shopping journey and fulfilment browser E2E on desktop/mobile	Auth journey only
REQUIRED	WCAG 2.2 AA audit including keyboard, zoom/reflow and screen reader	Not performed
REQUIRED	CSP, TLS/proxy settings, secret handling and security headers verified in production	Partial
REQUIRED	SLO dashboards, alerts, runbooks, rollback and incident ownership	Not present
REQUIRED	Load/capacity test and query review at expected peak traffic	Not performed
REQUIRED	Product structured data, sitemap, robots, canonicals and field Web Vitals	Not present

DECISION

NO-GO for broad public production. CONDITIONAL GO for a controlled COD pilot after dependency, recovery-email, PII policy, backup/restore and durable-notification blockers are closed.

Suggested re-review trigger

Repeat this audit after the 0-30 day release-safety work and before introducing online payments. Add a focused threat model, accessibility audit, performance field test and data-protection review at that point.

12 Research sources and audit references

W3C. Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/>

W3C WAI. What is new in WCAG 2.2. <https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/>

OWASP. API Security Top 10 - 2023. <https://owasp.org/API-Security/editions/2023/en/0x03-introduction/>

OWASP. Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

PCI SSC. PCI DSS v4.0.1 SAQ A ecommerce script eligibility FAQ. <https://www.pcisecuritystandards.org/faqs/1588/>

Next.js. Production checklist. <https://nextjs.org/docs/app/guides/production-checklist>

Next.js. Server and Client Components. <https://nextjs.org/docs/app/getting-started/server-and-client-components>

Google Search. Product structured data. <https://developers.google.com/search/docs/appearance/structured-data/product>

web.dev. Web Vitals. <https://web.dev/articles/vitals>

Google SRE. Service Level Objectives. <https://sre.google/sre-book/service-level-objectives/>

AWS. Well-Architected Reliability Pillar. <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html>

Stripe. Hosted Checkout. <https://docs.stripe.com/payments/checkout>

Stripe. Webhook security, retries and duplicate handling. <https://docs.stripe.com/webhooks>

Baymard. Cart abandonment and checkout usability research. <https://baymard.com/blog/ecommerce-checkout-usability-report-and-benchmark>

GitHub Advisory. js-yaml exponential parsing DoS. <https://github.com/advisories/GHSA-pm4m-ph32-ghv5>

GitHub Advisory. PostCSS XSS advisory. <https://github.com/advisories/GHSA-qx2v-qp2m-jg93>

GitHub Advisory. PostCSS source map information disclosure. <https://github.com/advisories/GHSA-6g55-p6wh-862q>

GitHub Advisory. PostCSS path traversal advisory. <https://github.com/advisories/GHSA-r28c-9q8g-f849>

GitHub Advisory. sharp/libvips inherited vulnerabilities. <https://github.com/advisories/GHSA-f88m-g3jw-g9cj>

Repository references

Primary local evidence: package.json; dashboard/package.json; prisma/schema.prisma and migrations; src/main.ts; src/app.module.ts; src/auth; src/rbac; src/shop; src/observability; src/health; dashboard/src/app; dashboard/src/components; test; e2e; .github/workflows/ci.yml; README.md; TESTING.md.

Verification record

Executed on 3 August 2026: npm run lint; npm run typecheck; npm run test:unit; npm run test:frontend; npm run test:integration; npm run test:database; npx prisma validate; npm run build; npm run test:e2e; npm audit --omit=dev in both workspaces.